

Math Foundations of Machine Learning

Linear Regression

Gabriel Wendell Celestino Rocha

July 2025

Table of Contents

Problem Formulation

Parameter Estimation



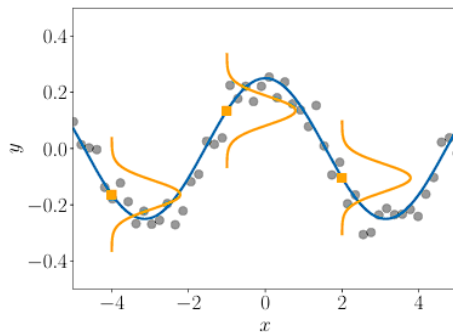
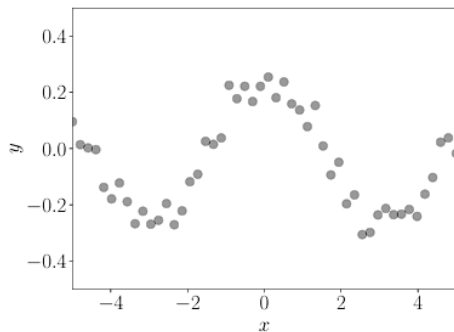
Problem Formulation

Introduction to Linear Regression

Goal: Find a function f mapping $\mathbf{x} \in \mathbb{R}^D \rightarrow y \in \mathbb{R}$

- Training inputs $\mathbf{x}_n \in \mathbb{R}^D$
- Observations $y_n = f(\mathbf{x}_n) + \epsilon$
- Noise ϵ : zero-mean Gaussian, i.i.d.

Objective: Generalize f beyond the training data



Applications and Challenges of Regression

Applications

- Time-series analysis (e.g., system identification)
- Control and robotics
- Optimization (e.g., line/global search)
- Deep Learning tasks:
Speech-to-text, image annotation,
video analysis
- Regression appears in supervised
and unsupervised learning

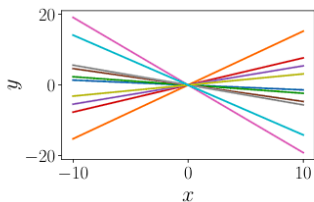
Key Modeling Challenges

- **Model Choice:** Which functions class? (e.g., polynomials)
- **Finding good parameters:** Fit via optimization (minimize loss)
- **Overfitting:** Too complex $\implies$ poor generalization
- **Loss and priors:** Link loss functions to statistical assumptions
- **Uncertainty:** Finite data induces uncertainty

Probabilistic Formulation

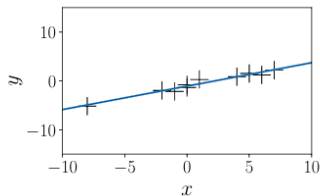
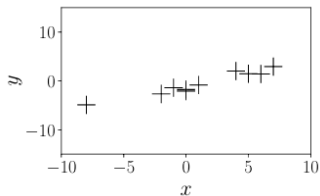
Linear Regression Assumptions

- We use **parametric models**:
 $f(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}$
- **Assumptions**:
 1. Known noise variance σ^2
 2. Model is linear in parameters
- **Goal**: Estimate parameters $\boldsymbol{\theta}$



Probabilistic Formulation

- **Noise model**: $\epsilon \sim \mathcal{N}(0, \sigma^2)$
- **Data model**:
 1. $y = f(\mathbf{x}) + \epsilon$
 2. $p(y|\mathbf{x}) = \mathcal{N}(y|\mathbf{x}^T \boldsymbol{\theta}, \sigma^2)$
- Without noise $\implies$ Dirac delta function





Parameter Estimation

Parameter Estimation and MLE

Parameter Estimation

- Given training set $\mathcal{D} := \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, assuming independence and a Gaussian likelihood for regression:

$$p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) = \prod_{n=1}^N p(y_n|\mathbf{x}_n, \boldsymbol{\theta}) = \prod_{n=1}^N \mathcal{N}(y_n|\mathbf{x}_n^T \boldsymbol{\theta}, \sigma^2) \quad , \quad \begin{cases} \mathcal{X} := \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \\ \mathcal{Y} := \{y_1, \dots, y_N\} \end{cases}$$

Maximum Likelihood Estimation

- Goal:** Maximize $p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) \implies \boldsymbol{\theta}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} [p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta})]$
- Use log-likelihood to avoid numerical issues:

$$\begin{aligned} -\log [p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta})] &= -\log \left[\prod_{n=1}^N p(y_n|\mathbf{x}_n, \boldsymbol{\theta}) \right] = -\sum_{n=1}^N \log [p(y_n|\mathbf{x}_n, \boldsymbol{\theta})] \\ &\implies \log [p(y_n|\mathbf{x}_n, \boldsymbol{\theta})] = -\frac{1}{2\sigma^2} (y_n - \mathbf{x}_n^T \boldsymbol{\theta})^2 + \text{const.} \end{aligned}$$

Analytic MLE Solution

- The negative log-likelihood (ignoring the constant terms) is:

$$\mathcal{L}(\boldsymbol{\theta}) := \frac{1}{2\sigma^2} \sum_{n=1}^N \left(y_n - \mathbf{x}_n^T \boldsymbol{\theta} \right)^2 = \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 .$$

- We can find the global optimum by computing the gradient of $\mathcal{L}$:

$$\frac{d\mathcal{L}}{d\boldsymbol{\theta}} \equiv \nabla_{\boldsymbol{\theta}} \mathcal{L} = \frac{1}{\sigma^2} \left(-\mathbf{y}^T \mathbf{X} + \boldsymbol{\theta}^T \mathbf{X}^T \mathbf{X} \right) \in \mathbb{R}^{1 \times D}$$

- The MLE estimator $\boldsymbol{\theta}_{\text{ML}}$ solves $\nabla_{\boldsymbol{\theta}} \mathcal{L} = \mathbf{0}^T$ (necessary optimality condition):

$$\frac{d\mathcal{L}}{d\boldsymbol{\theta}} \equiv \nabla_{\boldsymbol{\theta}} \mathcal{L} = \mathbf{0}^T \iff \boldsymbol{\theta}_{\text{ML}} = \left(\mathbf{X}^T \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{y} .$$

Valid if $\mathbf{X}^T \mathbf{X}$ is invertible (positive definite), i.e., $\text{rank}(\mathbf{X}) = D$.

Feature Transformations and Nonlinearity

Linear regression offers a way to fit nonlinear functions within the regression framework. We can perform an arbitrary nonlinear transformation/map $\phi(\mathbf{x})$ and then linearly combine the components of this transformation.

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\phi^T(\mathbf{x})\boldsymbol{\theta}, \sigma^2) \iff y = \phi^T(\mathbf{x})\boldsymbol{\theta} + \epsilon = \sum_{k=0}^{K-1} \theta_k \phi_k(\mathbf{x}) + \epsilon$$

- $\phi : \mathbb{R}^D \rightarrow \mathbb{R}^K$ is a nonlinear transformation of the inputs $\mathbf{x}$
- $\phi_k : \mathbb{R}^D \rightarrow \mathbb{R}$ is the k^{th} component of the *feature vector* ϕ .

“Linear regression” only refers to “linear in the parameters”.

Polynomial Regression

We are concerned with a regression problem $y = \phi^T(x)\theta + \epsilon$, where $x \in \mathbb{R}$ and $\theta \in \mathbb{R}^K$. A transformation that is often used on this contexts is:

$$\phi(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{K-1}(x) \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \\ \vdots \\ x^{K-1} \end{bmatrix} \in \mathbb{R}^K,$$

$$\implies f(x) = \sum_{k=0}^{K-1} \theta_k x^k = \phi^T(x)\theta, \quad \theta = (\theta_0, \dots, \theta_{K-1})^T \in \mathbb{R}^K.$$

This means that we “lift” the original one-dimensional input space into a K -dimensional feature space consisting of all monomials x^k for $k = 0, \dots, K - 1$.

The Design Matrix

Let us now have a look at MLE of the parameters θ in the linear regression model.

Design Matrix

Consider training inputs $\mathbf{x}_n \in \mathbb{R}^D$ and targets $y_n \in \mathbb{R}$ for $n = 1, \dots, N$. We define the *feature matrix* (*design matrix*) as:

$$\Phi := \begin{bmatrix} \phi^T(\mathbf{x}_1) \\ \vdots \\ \phi^T(\mathbf{x}_N) \end{bmatrix} = \begin{bmatrix} \phi_0(\mathbf{x}_1) & \dots & \phi_{K-1}(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \dots & \phi_{K-1}(\mathbf{x}_2) \\ \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_N) & \dots & \phi_{K-1}(\mathbf{x}_N) \end{bmatrix} \in \mathbb{R}^{N \times K}, \quad \begin{cases} \Phi_{ij} = \phi_j(\mathbf{x}_i) \\ \phi_j : \mathbb{R}^D \rightarrow \mathbb{R} \end{cases}$$

Feature Matrix for Second-order Polynomials

For a second-order polynomial and N training points $\mathbf{x}_n \in \mathbb{R}^D$, for $n = 1, \dots, N$. Determine the feature matrix for this case.

Negative Log-Likelihood of Polynomial Regression

The negative log-likelihood for the linear regression model can be written as follows. Since both $\mathbf{X}$ and Φ are independent of the parameters θ that we wish to optimize, we arrive immediately at the MLE:

$$-\log [p(\mathcal{Y}|\mathcal{X}, \theta)] = \frac{1}{2\sigma^2} (\mathbf{y} - \Phi\theta)^T (\mathbf{y} - \Phi\theta) + \text{const.} \implies \theta_{\text{ML}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}.$$

- Increasing $K \implies$ more complex fits.

MLE on Sinusoidal Data

Consider a dataset of $N = 10$ pairs (x_n, y_n) , where $x_n \sim \mathcal{U}[-5, 5]$ and $y_n = \sin(x_n/5) + \cos(x_n) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.2^2)$.

Fit a polynomial of degree 4 using MLE, i.e., parameters θ_{ML} presented above. The function values $\phi^T(x_*) \theta_{\text{ML}}$ at any test location x_* .

Estimating the Noise Variance

Assume that the noise variance σ^2 is known. However, we can also use the principle of maximum likelihood estimation to obtain the maximum likelihood estimator σ_{ML}^2 for the noise variance. To do this, we follow the standard procedure:

1. Write down the log-likelihood;
2. Compute its derivative with respect to $\sigma^2 > 0$;
3. Set the derivative to 0 and solve.

$$\begin{aligned}\log [p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}, \sigma^2)] &= \sum_{n=1}^N \log \left[\mathcal{N} \left(y_n | \phi^T(\mathbf{x}_n) \boldsymbol{\theta}, \sigma^2 \right) \right] \\ \implies \sigma_{\text{ML}}^2 &= \frac{S}{N} = \frac{1}{N} \sum_{n=1}^N \left[y_n - \phi^T(\mathbf{x}_n) \boldsymbol{\theta} \right]^2\end{aligned}$$

Therefore, the MLE of the noise variance is the empirical mean of the squared distances between the noise-free function values $\phi^T(\mathbf{x}_n)\boldsymbol{\theta}$ and the corresponding noisy observations y_n at input locations $\mathbf{x}_n$.

Overfitting in Linear Regression

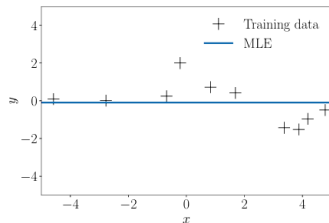
We just discussed how to use maximum likelihood estimation to fit linear models (e.g., polynomials) to data. We can evaluate the quality of the model by computing the error/loss incurred. One way of doing this is to compute the negative log-likelihood which we minimized to determine the maximum likelihood estimator.

- Low-degree $M \implies$ underfitting
- High-degree $M \implies$ overfitting
- Best generalization at optimal M

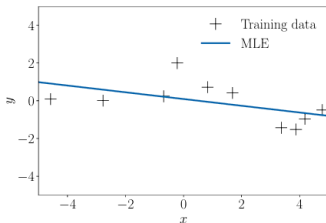
$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \|\mathbf{y} - \Phi\boldsymbol{\theta}\|^2} = \sqrt{\frac{1}{N} \sum_{n=1}^N [y_n - \phi^T(\mathbf{x}_n)\boldsymbol{\theta}]^2}.$$

The noise parameter σ^2 is not a free model parameter, we can ignore the scaling by $1/\sigma^2$, so that we end up with a squared-error-loss function.

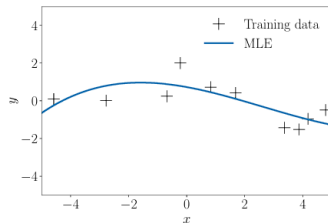
Selecting the Best Model



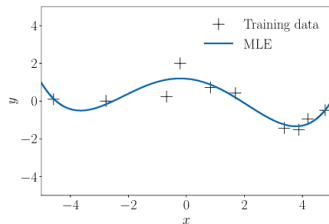
(a) $M = 0$



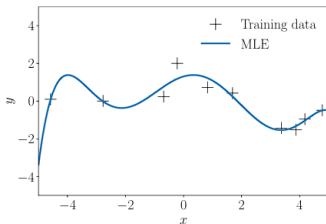
(b) $M = 1$



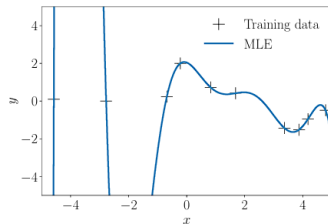
(c) $M = 3$



(d) $M = 4$



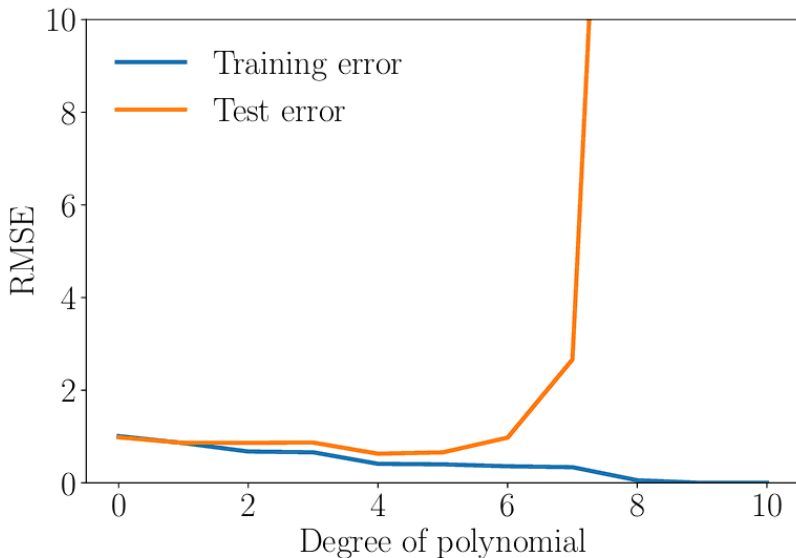
(e) $M = 6$



(f) $M = 9$

Selecting the Best Model

- Training error ↓ with complexity, test error 1 beyond optimal degree



Maximum A Posteriori Estimation (MAP)

We just saw that maximum likelihood estimation is prone to overfitting. We often observe that the magnitude of the parameter values becomes relatively large if we run into overfitting.

To mitigate the effect of huge parameter values, we can place a prior distribution $p(\boldsymbol{\theta})$ on the parameters, e.g., $p(\boldsymbol{\theta}) \propto \mathcal{N}(\mathbf{0}, 1)$. The prior distribution explicitly encodes what parameter values are plausible (before having seen any data).

The posterior over the parameters $\boldsymbol{\theta}$, given the training data $\mathcal{X}, \mathcal{Y}$, is obtained by applying Bayes' theorem as:

$$p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y}) = \frac{p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta})}{p(\mathcal{Y}|\mathcal{X})} \propto p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) ,$$

$$\implies \log [p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y})] = \log [p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta})] + \log [p(\boldsymbol{\theta})] + \text{const.}$$

Since the posterior explicitly depends on the parameter prior $p(\boldsymbol{\theta})$, the prior will have an effect on the parameter vector we find as the maximizer of the posterior.

MAP Estimation

To find the MAP estimate $\boldsymbol{\theta}_{\text{MAP}}$, we minimize the negative log-posterior distribution with respect to $\boldsymbol{\theta}$, i.e., we solve:

$$\boldsymbol{\theta}_{\text{MAP}} \in \arg \min_{\boldsymbol{\theta}} \{ -\log [\rho (\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta})] - \log [\rho (\boldsymbol{\theta})] \} .$$

The gradient of the negative log-posterior with respect to $\boldsymbol{\theta}$ is:

$$-\nabla_{\boldsymbol{\theta}} \log [\rho (\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y})] = -\nabla_{\boldsymbol{\theta}} \log [\rho (\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta})] - \nabla_{\boldsymbol{\theta}} \log [\rho (\boldsymbol{\theta})] .$$

With a (conjugate) Gaussian prior $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, b^2\mathbf{I})$ on the parameters $\boldsymbol{\theta}$, the negative log-posterior for the linear regression setting, we obtain the negative log posterior:

$$-\log [\rho (\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y})] = \frac{1}{2\sigma^2} (\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^T (\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \frac{1}{2b^2} (\boldsymbol{\theta}^T \boldsymbol{\theta}) + \text{const.}$$

$$\therefore \boxed{\boldsymbol{\theta}_{\text{MAP}} = \left(\boldsymbol{\Phi}^T \boldsymbol{\Phi} + \frac{\sigma^2}{b^2} \mathbf{I} \right)^{-1} \boldsymbol{\Phi}^T \mathbf{y}}$$

MAP Estimation

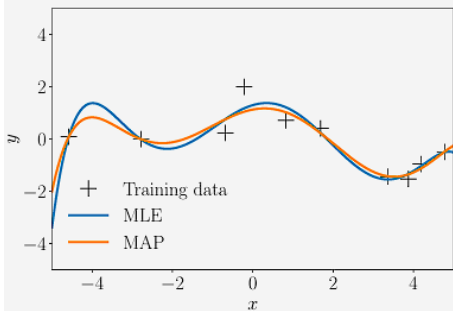
Comparing the MAP estimate in with the maximum likelihood MLE estimate, we see that the only difference between both solutions is the additional term $(\sigma^2/b^2) \mathbf{I}$ in the inverse matrix.

$$\boldsymbol{\theta}_{\text{MAP}} = \left(\boldsymbol{\Phi}^T \boldsymbol{\Phi} + \frac{\sigma^2}{b^2} \mathbf{I} \right)^{-1} \boldsymbol{\Phi}^T \mathbf{y} \quad | \quad \boldsymbol{\theta}_{\text{ML}} = \left(\boldsymbol{\Phi}^T \boldsymbol{\Phi} \right)^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

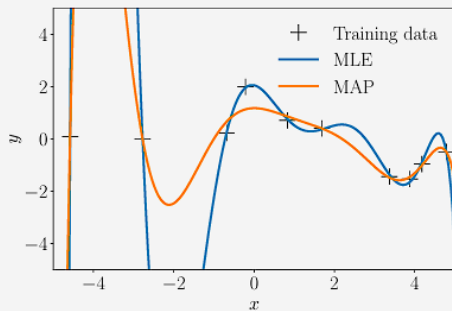
This term ensures that $\boldsymbol{\Phi}^T \boldsymbol{\Phi} + (\sigma^2/b^2) \mathbf{I}$ is symmetric and strictly positive definite (i.e., its inverse exists and the MAP estimate is the unique solution of a system of linear equations). Moreover, it reflects the impact of the regularizer.

MAP Estimation

The prior (regularizer) does not play a significant role for the low-degree polynomial, but keeps the function relatively smooth for higher-degree polynomials.



(a) Polynomials of degree 6.



(b) Polynomials of degree 8.

MAP Estimation as Regularization

Instead of placing a prior distribution on the parameters $\boldsymbol{\theta}$, it is also possible to mitigate the effect of overfitting by penalizing the amplitude of the parameter by means of *regularization*.

In *regularized least squares*, we consider the minimization of the loss function:

$$\arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}\|^2 + \lambda \|\boldsymbol{\theta}\|_2^2, \quad \lambda \geq 0.$$

- The first term is a *data-fit term* (also called *misfit term*), which is proportional to the negative log-likelihood
- The second term is called the *regularizer*, and the *regularization parameter* $\lambda \geq 0$ controls the “strictness” of the regularization.

The regularizer can be interpreted as a negative log-Gaussian prior, which we use in MAP estimation. More specifically, with a Gaussian prior $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, b^2\mathbf{I})$, we obtain the negative log-Gaussian prior:

$$-\log [p(\boldsymbol{\theta})] = \frac{1}{2b^2} \|\boldsymbol{\theta}\|_2^2 + \text{const.}$$

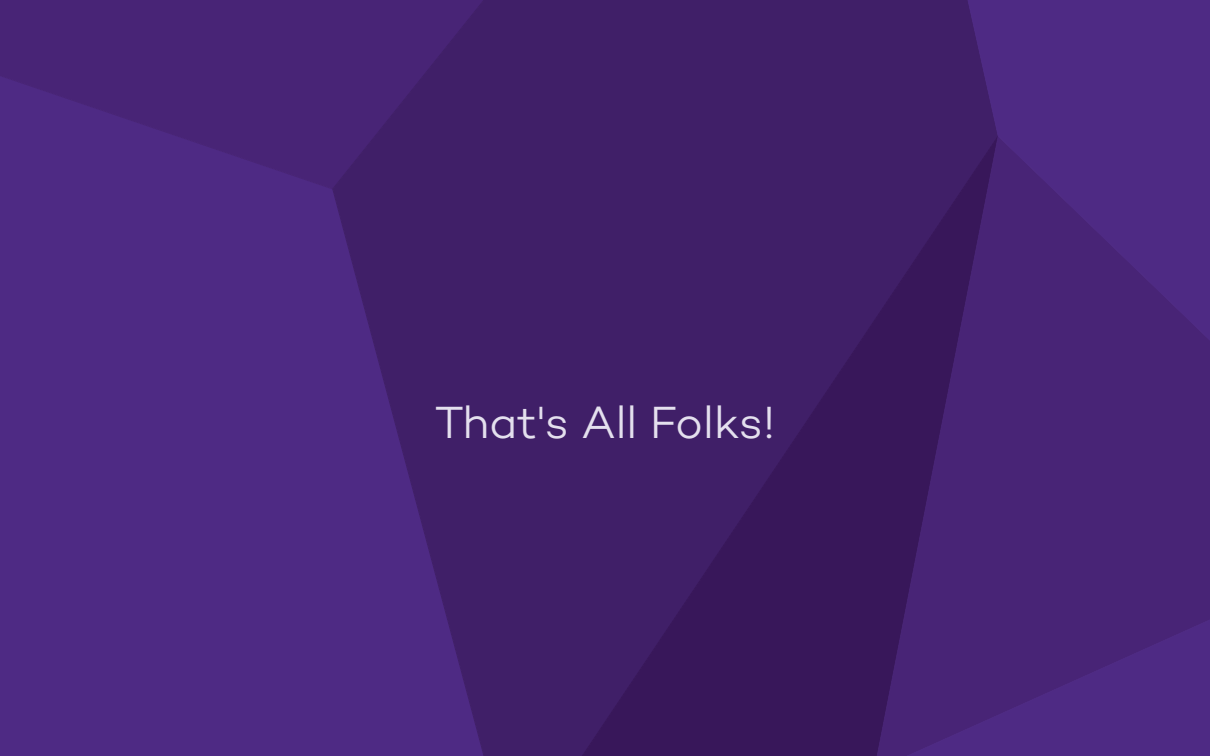
MAP Estimation as Regularization

Given that the regularized least-squares loss function consists of terms that are closely related to the negative log-likelihood plus a negative log-prior, it is not surprising that, when we minimize this loss, we obtain a solution that closely resembles the MAP estimate.

More specifically, minimizing the regularized least-squares loss function yields:

$$\boldsymbol{\theta}_{\text{RLS}} = \left(\boldsymbol{\Phi}^T \boldsymbol{\Phi} + \lambda \mathbf{I} \right)^{-1} \boldsymbol{\Phi}^T \mathbf{y},$$

which is identical to the MAP estimate in for $\lambda = \sigma^2/b^2$, where σ^2 is the noise variance and b^2 the variance of the (isotropic) Gaussian prior $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, b^2 \mathbf{I})$.

The background consists of several overlapping triangles in various shades of purple, creating a complex geometric pattern. The colors range from a deep, dark purple to a lighter, more vibrant shade.

That's All Folks!